

Distributed Financial Risk Analytics System

PySpark-Based Fraud Detection & Risk Analytics

Distributed Computing • Fraud Analytics • Cloud Deployment

Deesha / 2310990067

B.Tech CSE – Fintech Specialization



Problem Statement & Objectives



Core Challenges

- Traditional systems struggle to process large-scale transaction data efficiently
- Delayed fraud detection increases financial risk and security challenges

Project Objectives

- Build a scalable fintech analytics system
- Use PySpark for fast transaction processing
- Generate risk scores for transactions
- Detect suspicious and fraudulent activity
- Visualize analytics using an interactive dashboard
- Deploy securely on AWS EC2 cloud platform

Technology Stack



Frontend

Streamlit — Interactive dashboard & real-time visualization layer



Backend

Python — Core application logic, data generation, and risk engine



Distributed Computing

PySpark — Parallel transaction processing across compute resources



Data & Visualization

Pandas, NumPy, Plotly — Data manipulation and interactive charts



Cloud Deployment

AWS EC2, VPC, Security Groups — Scalable, isolated cloud infrastructure

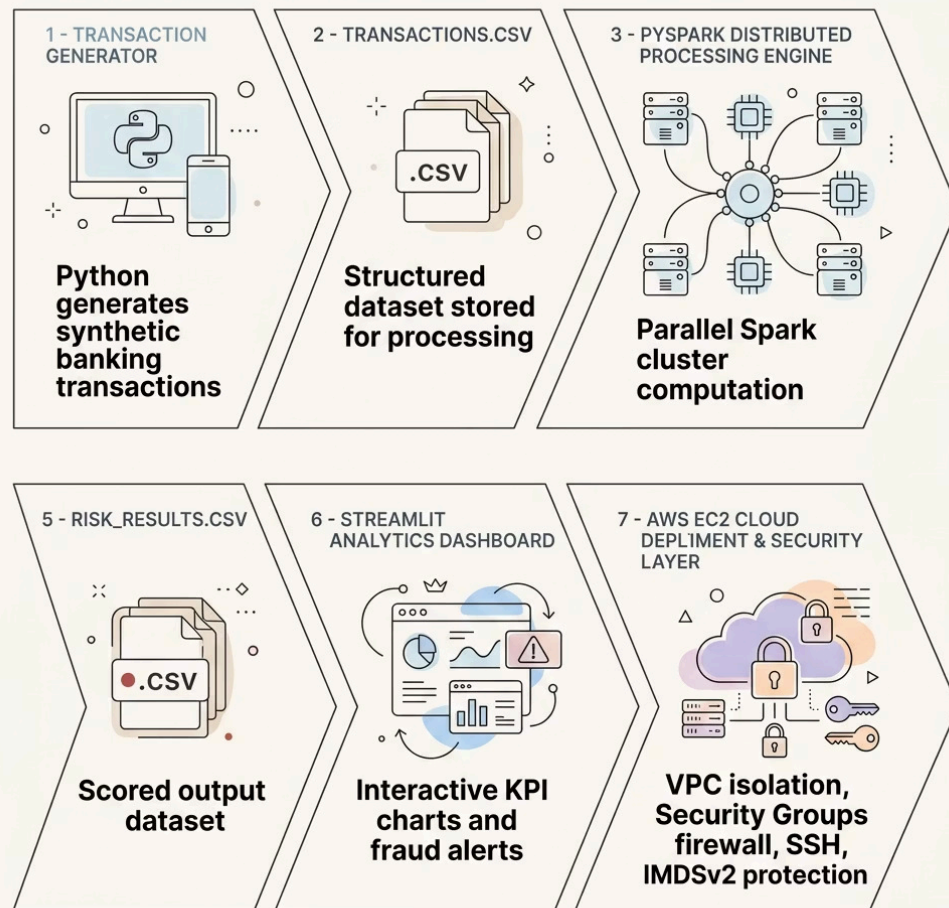


Security

SSH Authentication, IMDSv2 — Encrypted access and metadata protection

System Architecture

End-to-end enterprise fintech pipeline — from synthetic transaction generation through distributed PySpark processing, AI risk scoring, and secure AWS EC2 deployment with layered cloud security.

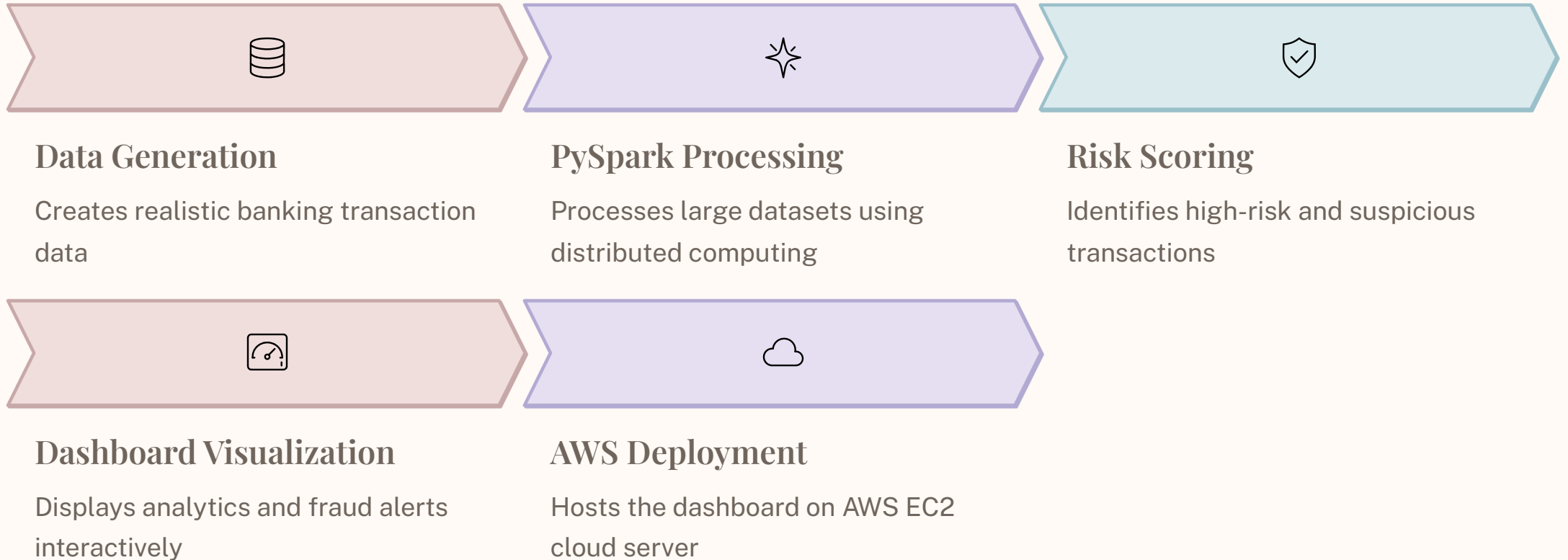


Architecture Highlights

- **Data Flow:** CSV ingestion → PySpark parallel processing → scored output
- **AI Risk Engine:** UDF-based scoring computes fraud risk 0–100 per transaction
- **Dashboard Layer:** Streamlit renders KPIs, charts, and real-time fraud alerts
- **Cloud Security:** VPC isolation, Security Groups, SSH authentication, IMDSv2

How the Project Works

Simple end-to-end workflow from transaction generation to cloud deployment.



PySpark Distributed Processing & Scalability

Traditional Python vs PySpark

Traditional Python (Sequential)

- Single-threaded execution
- Processes one transaction at a time
- Limited by single machine memory
- **Bottleneck:** Cannot scale beyond machine capacity
- **Processing time:** Linear with dataset size

PySpark (Distributed & Parallel)

- Multi-node cluster execution
- Processes millions of transactions simultaneously
- Distributed memory across cluster nodes
- **Scalable:** Add more nodes to increase capacity
- **Processing time:** Logarithmic with cluster size

PYSPARK IMPLEMENTATION

1

1. SparkSession

Entry point for all Spark operations

2

2. Distributed DataFrame

Data partitioned across cluster nodes

3

3. Parallel Execution

Tasks run simultaneously on multiple executors

4

4. UDF Risk Scoring

Custom fraud logic applied at scale

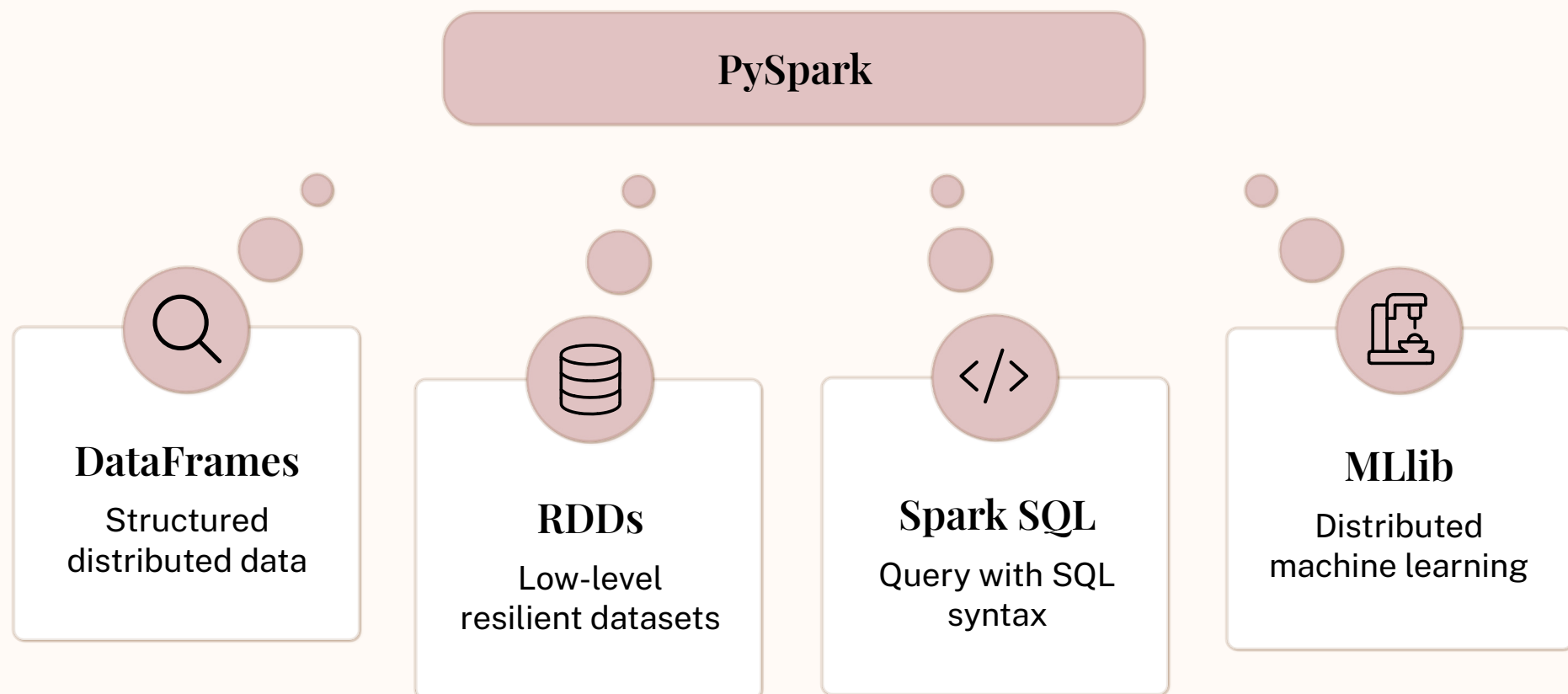
How PySpark Enhanced FinRisk

What is PySpark?

PySpark enables distributed and parallel processing of large datasets.

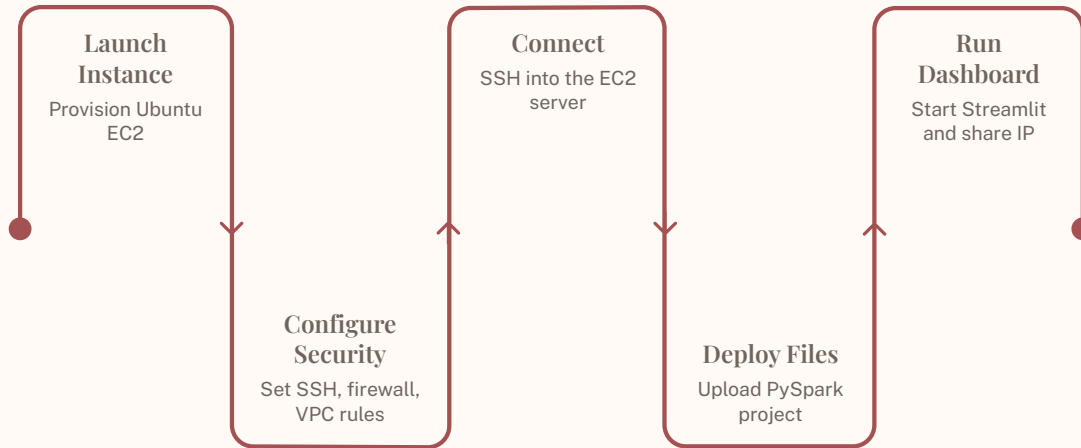
How FinRisk Uses PySpark

- Processes large financial transaction datasets
- Computes customer risk scores in parallel
- Powers scalable analytics on AWS EC2



AWS EC2 Deployment & Cloud Hosting

Deployment Workflow

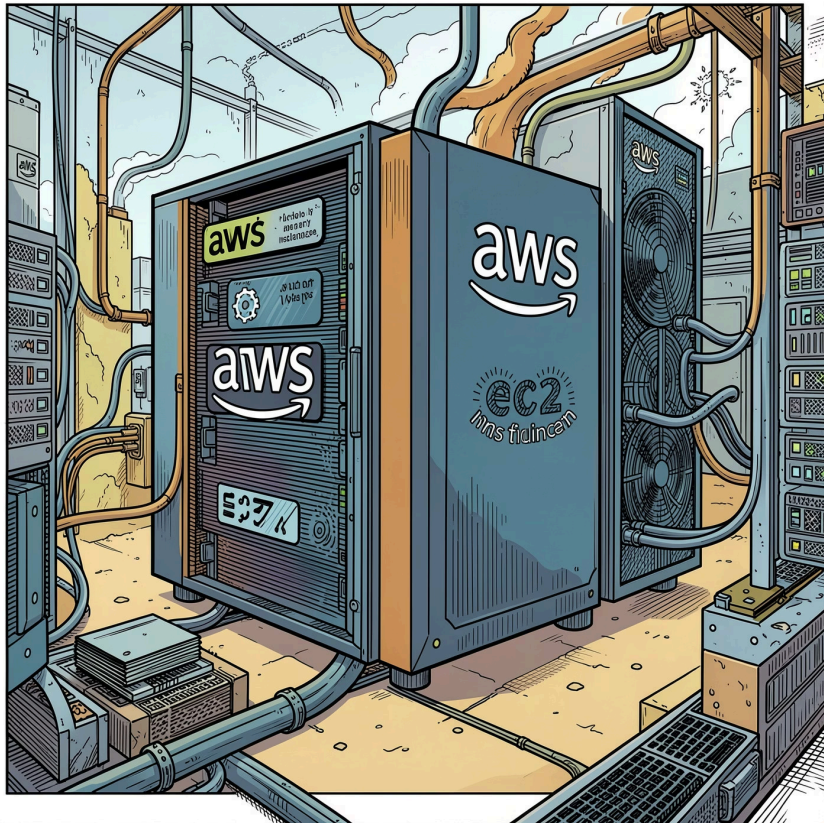


Provision the EC2 instance, secure access, and launch the dashboard remotely.

Key Deployment Details

- Ubuntu EC2 instance
- t2.medium instance type
- SSH key auth and security groups
- IMDSv2 enabled
- AWS VPC and subnet
- PySpark analytics and Streamlit dashboard

Cloud Security Implementation



VPC Network Isolation

Keeps EC2 instance in a secure private AWS network

Security Groups (Firewall)

Controls traffic — allows only SSH (port 22) and Streamlit (port 8501)

SSH Key Authentication

Secure login without passwords

Controlled Port Access

Only required ports enabled; all others blocked

IMDSv2 Protection

Extra security for EC2 metadata

Results, Conclusion & Future Scope

✦ KEY RESULTS

- Fast transaction processing using PySpark
- Fraud alerts generated successfully
- Interactive analytics dashboard created
- Secure deployment on AWS EC2

✓ CONCLUSION

PySpark, fraud analytics, and AWS cloud deployment were successfully integrated to build a scalable financial risk analysis system.

🚀 FUTURE SCOPE

- Real-time Kafka streaming
- Machine learning-based fraud detection
- Docker & Kubernetes deployment
- AWS S3 cloud storage integration